

FRE6871 R in Finance

Lecture#7, Fall 2025

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

December 8, 2025



NYU

**TANDON SCHOOL
OF ENGINEERING**

Package *Rcpp* for Calling C++ Programs from R

The package *Rcpp* allows calling C++ functions from R, by compiling the C++ code and creating R functions.

Rcpp functions are R functions that were compiled from C++ code using package *Rcpp*.

Rcpp functions are much faster than code written in R, so they're suitable for large numerical calculations.

The package *Rcpp* relies on *Rtools* for compiling the C++ code:

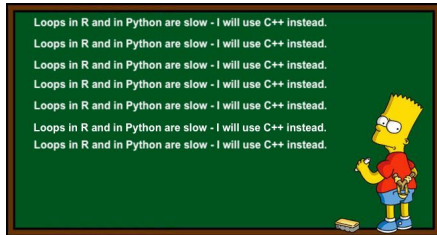
<https://cran.r-project.org/bin/windows/Rtools/>

You can learn more about the package *Rcpp* here:

<http://adv-r.had.co.nz/Rcpp.html>

<http://www.rcpp.org/>

<http://gallery.rcpp.org/>



```
> # Verify that Rtools or XCode are working properly:
> devtools::find_rtools() ## Under Windows
> devtools::has_devel()
> # Install the packages Rcpp and RcppArmadillo
> install.packages(c("Rcpp", "RcppArmadillo"))
> # Load package Rcpp
> library(Rcpp)
> # Get documentation for package Rcpp
> # Get short description
> packageDescription("Rcpp")
> # Load help page
> help(package="Rcpp")
> # List all datasets in "Rcpp"
> data(package="Rcpp")
> # List all objects in "Rcpp"
> ls("package:Rcpp")
> # Remove Rcpp from search path
> detach("package:Rcpp")
```

Function `cppFunction()` for Compiling C++ code

The function `cppFunction()` compiles C++ code into an R function.

The function `cppFunction()` creates an R function only for the current R session, and it must be recompiled for every new R session.

The function `sourceCpp()` compiles C++ code contained in a file into R functions.

```
> # Define Rcpp function
> Rcpp::cppFunction("
+   int times_two(int x)
+   { return 2 * x;}
+   ") ## end cppFunction
> # Run Rcpp function
> times_two(3)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts
> # Multiply two numbers
> mult_rcpp(2, 3)
> mult_rcpp(1:3, 6:4)
> # Multiply two vectors
> mult_vec_rcpp(2, 3)
> mult_vec_rcpp(1:3, 6:4)
```

Performing Loops in *Rcpp Sugar*

Loops written in *Rcpp* can be two orders of magnitude faster than loops in R!

Rcpp Sugar allows using R-style vectorized syntax in *Rcpp* code.

```
> # Define Rcpp function with loop
> Rcpp::cppFunction("
+ double inner_mult(NumericVector x, NumericVector y) {
+   int xsize = x.size();
+   int ysize = y.size();
+   if (xsize != ysize) {
+     return 0;
+   } else {
+     double total = 0;
+     for(int i = 0; i < xsize; ++i) {
+       total += x[i] * y[i];
+     }
+     return total;
+   }
+ }") ## end cppFunction
> # Run Rcpp function
> inner_mult(1:3, 6:4)
> inner_mult(1:3, 6:3)
> # Define Rcpp Sugar function with loop
> Rcpp::cppFunction("
+ double inner_sugar(NumericVector x, NumericVector y) {
+   return sum(x * y);
+ }") ## end cppFunction
> # Run Rcpp Sugar function
> inner_sugar(1:3, 6:4)
> inner_sugar(1:3, 6:3)
```

```
> # Define R function with loop
> inner_mult_r <- function(x, y) {
+   sumv <- 0
+   for(i in 1:NROW(x)) {
+     sumv <- sumv + x[i] * y[i]
+   }
+   sumv
+ } ## end inner_mult_r
> # Run R function
> inner_mult_r(1:3, 6:4)
> inner_mult_r(1:3, 6:3)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=inner_mult_r(1:10000, 1:10000),
+   innerp=1:10000 %*% 1:10000,
+   Rcpp=inner_mult(1:10000, 1:10000),
+   sugar=inner_sugar(1:10000, 1:10000),
+   times=10))[, c(1, 4, 5)]
```

Simulating Ornstein-Uhlenbeck Process Using *Rcpp*

Simulating the Ornstein-Uhlenbeck Process in *Rcpp* is about 30 times faster than in R!

```
> # Define Ornstein-Uhlenbeck function in R
> sim_our <- function(nrows=1000, priceq=5.0,
+   volat=0.01, theta=0.01) {
+   retp <- numeric(nrows)
+   pricev <- numeric(nrows)
+   pricev[1] <- priceq
+   for (i in 2:nrows) {
+     retp[i] <- theta*(priceq - pricev[i-1]) + volat*rnorm(1)
+     pricev[i] <- pricev[i-1] + retp[i]
+   } ## end for
+   pricev
+ } ## end sim_our
> # Simulate Ornstein-Uhlenbeck process in R
> priceq <- 5.0; sigmav <- 0.01
> thetav <- 0.01; nrows <- 1000
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection") ##
> ousim <- sim_our(nrows, priceq=priceq, volat=sigmav, theta=teta
```

```
> # Define Ornstein-Uhlenbeck function in Rcpp
> Rcnp::cppFunction("
+ NumericVector sim_oucnp(double priceq,
+   double volat,
+   double thetav,
+   NumericVector innov) {
+   int nrows = innov.size();
+   NumericVector pricev(nrows);
+   NumericVector retv(nrows);
+   pricev[0] = priceq;
+   for (int it = 1; it < nrows; it++) {
+     retv[it] = thetav*(priceq - pricev[it-1]) + volat*innov[it-1];
+     pricev[it] = pricev[it-1] + retv[it];
+   } // end for
+   return pricev;
+ }") ## end cppFunction
> # Simulate Ornstein-Uhlenbeck process in Rcpp
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection") ##
> oucnp <- sim_oucnp(priceq=priceq,
+   volat=sigmav, thetav=teta, innov=rnorm(nrows))
> all.equal(ousim, oucnp)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=sim_our(nrows, priceq=priceq, volat=sigmav, theta=teta),
+   Rcnp=sim_oucnp(priceq=priceq, volat=sigmav, theta=teta, innov=
+   times=10))[, c(1, 4, 5)])
```

Rcpp Attributes

Rcpp attributes are instructions for the C++ compiler, embedded in the *Rcpp* code as C++ comments, and preceded by the `///` symbol.

The `Rcpp::depends` attribute specifies additional C++ library dependencies.

The `Rcpp::export` attribute specifies that a function should be exported to R, where it can be called as an R function.

Only functions which are preceded by the `Rcpp::export` attribute are exported to R.

The function `sourceCpp()` compiles C++ code contained in a file into R functions.

```
> # Source Rcpp function for Ornstein-Uhlenbeck process from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts,}
> # Simulate Ornstein-Uhlenbeck process in Rcpp
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection") ## Reset random numbers
> oucpp <- sim_oucpp(priceq=priceq,
+   volat=sigmav,
+   theta=thetav,
+   innov=rnorm(nrows))
> all.equal(ousim, oucpp)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=sim_our(nrows, priceq=priceq, volat=sigmav, theta=thetav),
+   Rcpp=sim_oucpp(priceq=priceq, volat=sigmav, theta=thetav, innov=rnorm(nrows)),
+   times=10))[, c(1, 4, 5)]
```

```
// Rcpp header with information for C++ compiler
#include <Rcpp.h> // include Rcpp C++ header files
using namespace Rcpp; // use Rcpp C++ namespace

// The function sim_oucpp() simulates an Ornstein-Uhlenbeck process
// export the function roll_maxmin() to R
// [[Rcpp::export]]
NumericVector sim_oucpp(double priceq,
                        double volat,
                        double thetav,
                        NumericVector innov) {
  int(nrows = innov.size());
  NumericVector pricev*nrows);
  NumericVector retp*nrows);
  pricev[0] = priceq;
  for (int it = 1; it < nrows; it++) {
    retp[it] = thetav*(priceq - pricev[it-1]) + volat*innov[it];
    pricev[it] = pricev[it-1] + retp[it];
  } // end for
  return pricev;
} // end sim_oucpp
```

Generating Random Numbers Using Logistic Map in *Rcpp*

The *logistic map* in *Rcpp* is about seven times faster than the loop in R, and even slightly faster than the standard `runif()` function in R!

```
> # Calculate uniformly distributed pseudo-random sequence
> unifun <- function(seedv, nrows=10) {
+   datav <- numeric(nrows)
+   datav[1] <- seedv
+   for (i in 2:nrows) {
+     datav[i] <- 4*datav[i-1]*(1-datav[i-1])
+   } ## end for
+   acos(1-2*datav)/pi
+ } ## end unifun
>
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> # Microbenchmark Rcpp code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=runif(1e5),
+   rloop=unifun(0.3, 1e5),
+   Rcpp=unifuncpp(0.3, 1e5),
+   times=10))[, c(1, 4, 5)]
```

```
// Rcpp header with information for C++ compiler
#include <Rcpp.h> // include Rcpp C++ header files
using namespace Rcpp; // use Rcpp C++ namespace

// This is a simple example of exporting a C++ function
// You can source this function into an R session using
// function Rcpp::sourceCpp()
// (or via the Source button on the editor toolbar).
// Learn more about Rcpp at:
//
//   http://www.rcpp.org/
//   http://adv-r.had.co.nz/Rcpp.html
//   http://gallery.rcpp.org/

// function unifun() produces a vector of
// uniformly distributed pseudo-random numbers
// [[Rcpp::export]]
NumericVector unifuncpp(double seedv, int(nrows) {
  // define pi
  static const double pi = 3.14159265;
  // allocate output vector
  NumericVector datav(nrows);
  // initialize output vector
  datav[0] = seedv;
  // perform loop
  for (int i=1; i < nrows; ++i) {
    datav[i] = 4*datav[i-1]*(1-datav[i-1]);
  } // end for
  // rescale output vector and return it
  return acos(1-2*datav)/pi;
}
```

Package *RcppArmadillo* for Fast Linear Algebra

The package *RcppArmadillo* allows calling from R the high-level *Armadillo* C++ linear algebra library.

Armadillo provides ease of use and speed, with syntax similar to *Matlab*.

RcppArmadillo functions are often faster than even compiled R functions, because they use better optimized C++ code:

<http://arma.sourceforge.net/speed.html>

You can learn more about *RcppArmadillo*:

<http://arma.sourceforge.net/>

<http://dirk.eddelbuettel.com/code/rcpp.armadillo.html>

<https://cran.r-project.org/web/packages/\emph{RcppArmadillo}/index.html>

<https://github.com/RcppCore/\emph{RcppArmadillo}>

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/script:
> vec1 <- runif(1e5)
> vec2 <- runif(1e5)
> inner_vec(vec1, vec2)
> vec1 %*% vec2
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h>
using namespace Rcpp;
using namespace arma;
// [[Rcpp::depends(RcppArmadillo)]]

// The function inner_vec() calculates the inner (dot) p
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inner_vec(arma::vec vec1, arma::vec vec2) {
  return arma::dot(vec1, vec2);
} // end inner_vec

// The function inner_mat() calculates the inner (dot) p
// with two vectors.
// It accepts pointers to the matrix and vectors, and re
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inner_mat(const arma::vec& vecv2, const arma::mat
  return arma::as_scalar(trans(vecv2) * (matv * vecv1));
} // end inner_mat

> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcpp = inner_vec(vec1, vec2),
+   rcode = (vec1 %*% vec2),
+   times=100))[, c(1, 4, 5)] ## end microbenchmark summary
> # Microbenchmark shows:
> # inner_vec() is several times faster than %*%, especially for lo
> #      expr      mean      median
> # 1 inner_vec 110.7067 110.4530
> # 2 rcode     585.5127 591.3575
```


Simulating ARIMA Processes Using RcppArmadillo

ARIMA processes can be simulated using *RcppArmadillo* even faster than by using the function `filter()`.

```
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts,
> # Define AR(2) coefficients
> coeff <- c(0.9, 0.09)
> nrows <- 1e4
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> innov <- rnorm(nrows)
> # Simulate ARIMA using filter()
> arimar <- filter(x=innov, filter=coeff, method="recursive")
> # Simulate ARIMA using sim_ar()
> innov <- matrix(innov)
> coeff <- matrix(coeff)
> arimav <- sim_ar(coeff, innov)
> all.equal(drop(arimav), as.numeric(arimar))
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcpp = sim_ar(coeff, innov),
+   filter = filter(x=innov, filter=coeff, method="recursive"),
+   times=100))[, c(1, 4, 5)] ## end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h> // include C++ header file for
using namespace arma; // use C++ namespace from Armadillo
// declare dependency on RcppArmadillo
// [[Rcpp::depends(RcppArmadillo)]]

// @export
// [[Rcpp::export]]
arma::vec sim_ar(const arma::vec& innov, const arma::vec&
  uword nrows = innov.n_elem;
  uword lookb = coeff.n_elem;
  arma::vec arimav(nrows);

  // startup period
  arimav(0) = innov(0);
  arimav(1) = innov(1) + coeff(lookb-1) * arimav(0);
  for (uword it = 2; it < lookb-1; it++) {
    arimav(it) = innov(it) + arma::dot(coeff.subvec(lookb-1, it-1),
  } // end for

  // remaining periods
  for (uword it = lookb; it < nrows; it++) {
    arimav(it) = innov(it) + arma::dot(coeff, arimav.subvec(0, it-1));
  } // end for

  return arimav;
} // end sim_arima
```

Fast Matrix Algebra Using *RcppArmadillo*

RcppArmadillo functions can be made even faster by operating on pointers to matrices and performing calculations in place, without copying large matrices.

RcppArmadillo functions can be compiled using the same *Rtools* as those for *Rcpp* functions:

<https://cran.r-project.org/bin/windows/Rtools/>

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> matv <- matrix(runif(1e5), nc=1e3)
> # Center matrix columns using apply()
> matd <- apply(matv, 2, function(x) (x-mean(x)))
> # Center matrix columns in place using Rcpp demeanr()
> demeanr(matv)
> all.equal(matd, matv)
> # Microbenchmark \emph{RcppArmadillo} code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode = (apply(matv, 2, mean)),
+   rcpp = demeanr(matv),
+   times=100))[, c(1, 4, 5)] ## end microbenchmark summary
> # Perform matrix inversion
> # Create random positive semi-definite matrix
> matv <- matrix(runif(25), nc=5)
> matv <- t(matv) %*% matv
> # Invert the matrix
> matrixinv <- solve(matv)
> inv_mat(matv)
> all.equal(matrixinv, matv)
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcode = solve(matv),
+   rcpp = inv_mat(matv),
+   times=100))[, c(1, 4, 5)] ## end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h> // include C++ header file for
using namespace arma; // use C++ namespace from Armadillo
// declare dependency on RcppArmadillo
// [[Rcpp::depends(RcppArmadillo)]]

// Examples of \emph{RcppArmadillo} functions below

// The function demeanr() calculates a matrix with centered
// It accepts a pointer to a matrix and operates on the
// It returns the number of columns of the input matrix.
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
int demeanr(arma::mat& matv) {
  for (uword i = 0; i < matv.n_cols; i++) {
    matv.col(i) -= arma::mean(matv.col(i));
  } // end for
  return matv.n_cols;
} // end demeanr

// The function inv_mat() calculates the inverse of symmetric
// definite matrix.
// It accepts a pointer to a matrix and operates on the
// It returns the number of columns of the input matrix.
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inv_mat(arma::mat& matv) {
  matv = arma::inv_sympd(matv);
  return matv.n_cols;
} // end inv_mat
```

Fast Correlation Matrix Inverse Using RcppArmadillo

RcppArmadillo can be used to quickly calculate the reduced inverse of correlation matrices.

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp("/Users/jerzy/Develop/lecture_slides/scripts/High")
> # Calculate matrix of random returns
> matv <- matrix(rnorm(300), nc=5)
> # Reduced inverse of correlation matrix
> dimax <- 4
> cormat <- cor(matv)
> eigend <- eigen(cormat)
> invmat <- eigend$vectors[, 1:dimax] %*%
+   (t(eigend$vectors[, 1:dimax]) / eigend$values[1:dimax])
> # Reduced inverse using \emph{RcppArmadillo}
> invarma <- calc_inv(cormat, dimax=dimax)
> all.equal(invmat, invarma)
> # Microbenchmark \emph{RcppArmadillo} code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode = {eigend <- eigen(cormat)
+   eigend$vectors[, 1:dimax] %*% (t(eigend$vectors[, 1:dimax]) / eig
+   rcpp = calc_inv(cormat, dimax=dimax),
+   times=100))[, c(1, 4, 5)] ## end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
// include Rcpp C++ header files
using namespace std;
using namespace Rcpp; // use Rcpp C++ namespace
using namespace arma;

//' @export
// [[Rcpp::export]]
arma::mat calc_inv(const arma::mat& matv,
                   arma::uword dimax = 0, // Max number
                   double eigen_thresh = 0.01) { // Thresh

    // Allocate SVD variables
    arma::vec svdval; // Singular values
    arma::mat svdu, svdv; // Singular matrices
    // Calculate the SVD
    arma::svd(svdu, svdval, svdv, tseries);
    // Calculate the number of non-small singular values
    arma::uword svdnum = arma::sum(svdval > eigen_thresh)*a

    // If no regularization then set dimax to (svdnum - 1)
    if (dimax == 0) {
        // Set dimax
        dimax = svdnum - 1;
    } else {
        // Adjust dimax
        dimax = stdev::min(dimax - 1, svdnum - 1);
    } // end if

    // Remove all small singular values
    svdval = svdval.subvec(0, dimax);
    svdu = svdu.cols(0, dimax);
    svdv = svdv.cols(0, dimax);

    // Calculate the reduced inverse from the SVD decompos
    return svdv*arma::diagmat(1/svdval)*svdu.t();
```

Portfolio Optimization Using RcppArmadillo

Fast portfolio optimization using matrix algebra can be implemented using RcppArmadillo.

```
// Fast portfolio optimization using matrix algebra and \emph{RcppArmadillo}
arma::vec calc_weights(const arma::mat& returns, // Asset returns
                      Rcpp::List controll) { // List of portfolio optimization parameters

    // Apply different calculation methods for weights
    switch(calc_method(method)) {
    case methodenum::maxsharpe: {
        // Mean returns of columns
        arma::vec colmeans = arma::trans(arma::mean(returns, 0));
        // Shrink colmeans to the mean of returns
        colmeans = ((1-alpha)*colmeans + alpha*arma::mean(colmeans));
        // Calculate weights using reduced inverse
        weights = calc_inv(covmat, dimax, eigen_thresh)*colmeans;
        break;
    } // end maxsharpe
    case methodenum::maxsharpemed: {
        // Median returns of columns
        arma::vec colmeans = arma::trans(arma::median(returns, 0));
        // Shrink colmeans to the median of returns
        colmeans = ((1-alpha)*colmeans + alpha*arma::median(colmeans));
        // Calculate weights using reduced inverse
        weights = calc_inv(covmat, dimax, eigen_thresh)*colmeans;
        break;
    } // end maxsharpemed
    case methodenum::minvarlin: {
        // Minimum variance weights under linear constraint
        // Multiply reduced inverse times unit vector
        weights = calc_inv(covmat, dimax, eigen_thresh)*arma::ones(ncols);
        break;
    } // end minvarlin
    case methodenum::minvarquad: {
        // Minimum variance weights under quadratic constraint
        // Calculate highest order principal component
        arma::vec eigenval;
        arma::mat eigenvec;
```

Strategy Backtesting Using RcppArmadillo

Fast backtesting of strategies can be implemented using *RcppArmadillo*.

```
arma::mat roll_portf(const arma::mat& excess, // Asset excess returns
                    const arma::mat& returns, // Asset returns
                    Rcpp::List controll, // List of portfolio optimization model parameters
                    arma::uvec startp, // Start points
                    arma::uvec endd, // End points
                    double lambdaf = 0.0, // Decay factor for averaging the portfolio weights
                    double coeff = 1.0, // Multiplier of strategy returns
                    double bidask = 0.0) { // The bid-ask spread

    double lambdai = 1-lambdaf;
    arma::uword nweights = returns.n_cols;
    arma::vec weights(nweights, fill::zeros);
    arma::vec weights_past = ones(nweights)/stdev::sqrt(nweights);
    arma::mat pnls = zeros(returns.n_rows, 1);

    // Perform loop over the end points
    for (arma::uword it = 1; it < endd.size(); it++) {
        // cout << "it: " << it << endl;
        // Calculate the portfolio weights
        weights = coeff*calc_weights(excess.rows(startp(it-1), endd(it-1)), controll);
        // Calculate the weights as the weighted sum with past weights
        weights = lambdai*weights + lambdaf*weights_past;
        // Calculate out-of-sample returns
        pnls.rows(endd(it-1)+1, endd(it)) = returns.rows(endd(it-1)+1, endd(it))*weights;
        // Add transaction costs
        pnls.row(endd(it-1)+1) -= bidask*sum(abs(weightv - weights_past))/2;
        // Copy the weights
        weights_past = weights;
    } // end for

    // Return the strategy pnls
    return pnls;
} // end roll_portf
```

Package *reticulate* for Running Python from RStudio

The package *reticulate* allows running Python functions and scripts from RStudio.

The package *reticulate* relies on Python for interpreting the Python code.

You must set your Global Options in RStudio to your Python executable, for example:

/Library/Frameworks/Python.framework/Versions/3.10/bin/python3.10

You can learn more about the package *reticulate* here:

<https://rstudio.github.io/reticulate/>

```
> # Install package reticulate
> install.packages("reticulate")
> # Start Python session
> reticulate::repl_python()
> # Exit Python session
> exit
```

Running Python Under *reticulate*

```
"""
Script for loading OHLC data from a CSV file and plotting a candlestick plot.
"""
# Import packages
import pandas as pd
import numpy as np
import plotly.graph_objects as go
# Load OHLC data from csv file - the time index is formatted inside read_csv()
symbol = "SPY"
range = "day"
filename = "/Users/jerzy/Develop/data/" + symbol + "_" + range + ".csv"
ohlc = pd.read_csv(filename)
datev = ohlc.Date
# Calculate log stock prices
ohlc[["Open", "High", "Low", "Close"]] = np.log(ohlc[["Open", "High", "Low", "Close"]])
# Calculate moving average
lookback = 55
closep = ohlc.Close
pricema = closep.ewm(span=lookback, adjust=False).mean()
# Plotly simple candlestick with moving average
# Create empty graph object
plotfig = go.Figure()
# Add trace for candlesticks
plotfig = plotfig.add_trace(go.Candlestick(x=datev,
    open=ohlc.Open, high=ohlc.High, low=ohlc.Low, close=ohlc.Close,
    name=symbol+" Log OHLC Prices", showlegend=False))
# Add trace for moving average
plotfig = plotfig.add_trace(go.Scatter(x=datev, y=pricema,
    name="Moving Average", line=dict(color="blue")))
# Customize plot
plotfig = plotfig.update_layout(title=symbol + " Log OHLC Prices",
    title_font_size=24, title_font_color="blue", yaxis_title="Price",
    font_color="black", font_size=18, xaxis_rangeslider_visible=False)
# Customize legend
plotfig = plotfig.update_layout(legend=dict(x=0.2, y=0.9, traceorder="normal",
    itemsizing="constant", font=dict(family="sans-serif", size=18, color="blue")))
# Render the plot
plotfig.show()
```

Reading Data From Excel Files

The package *readxl* reads data from Excel spreadsheet files into R.

The function `read_excel()` reads a single sheet (tab) from an Excel file.

The function `read_xlsx()` reads a single sheet (tab) from an Excel file in .xlsx format.

The functions from package *readxl* return a type of *data frame* called a *tibble* object.

The *tibble* classes `tbl` and `tbl_df` are derived from the *data frame* class `data.frame`.

tibble objects are also used by the package *dplyr*.

DataCamp offers a [Tutorial on Importing Excel Files into R](#).

```
> # Install and load package readxl
> install.packages("readxl")
> library(readxl)
> dirn <- "/Users/jerzy/Develop/lecture_slides/data"
> filev <- file.path(dirn, "multi_tabs.xlsx")
> # Read a time series from first sheet of xlsx file
> tibblev <- readxl::read_xlsx(filev)
> class(tibblev)
> # Coerce POSIXct dates into Date class
> class(tibblev$Dates)
> tibblev$Dates <- as.Date(tibblev$Dates)
> # Some columns are character strings
> sapply(tibblev, class)
> sapply(tibblev, is.character)
> # Coerce columns with strings to numeric
> listv <- lapply(tibblev, function(x) {
+   if (is.character(x))
+     as.numeric(x)
+   else
+     x
+ }) ## end lapply
> # Coerce list into xts time series
> xtsv <- xts::xts(do.call(cbind, listv)[, -1], listv[[1]])
> class(xtsv); dim(xtsv)
> # Replace NA values with the most recent non-NA values
> sum(is.na(xtsv))
> xtsv <- zoo::na.locf(xtsv, na.rm=FALSE)
> xtsv <- zoo::na.locf(xtsv, fromLast=TRUE)
```


Reading Multiple Sheets From Excel Files

The function `readxl::excel_sheets()` returns a vector of character strings with the names of all the sheets in an Excel spreadsheet.

The package *readxl* reads data from Excel spreadsheet files into R.

The function `read_excel()` reads a single sheet (tab) from an Excel file.

The function `read_xlsx()` reads a single sheet (tab) from an Excel file in .xlsx format.

The functions from package *readxl* return a type of *data frame* called a *tibble* object.

The *tibble* classes `tbl` and `tbl_df` are derived from the *data frame* class `data.frame`.

tibble objects are also used by the package *dplyr*.

```
> # Read names of all the sheets in an Excel spreadsheet
> namev <- readxl::excel_sheets(filev)
> # Read all the sheets from an Excel spreadsheet
> sheetv <- lapply(namev, read_xlsx, path=filev)
> names(sheetv) <- namev
> # sheetv is a list of tibbles
> sapply(sheetv, class)
> # Create function to coerce tibble to xts
> to_xts <- function(tibblev) {
+   tibblev$Dates <- as.Date(tibblev$Dates)
+   ## Coerce columns with strings to numeric
+   listv <- lapply(tibblev, function(x) {
+     if (is.character(x))
+       as.numeric(x)
+     else
+       x
+   }) ## end lapply
+   ## Coerce list into xts series
+   xts::xts(do.call(cbind, listv)[-1], listv$Dates)
+ } ## end to_xts
> # Coerce list of tibbles to list of xts
> class(sheetv)
> sheetv <- lapply(sheetv, to_xts)
> sapply(sheetv, class)
> # Replace NA values with the most recent non-NA values
> sapply(sheetv, function(xtsv) sum(is.na(xtsv)))
> sheetv <- lapply(sheetv, zoo::na.locf, na.rm=FALSE)
> sheetv <- lapply(sheetv, zoo::na.locf, fromLast=TRUE)
```

Performing Calculations in Excel Using R

Excel can run R using either VBA scripts, or through a *COM* interface (available on *Windows* only).

R can perform calculations and export its output to Excel files, or it can modify Excel files (requires packages using Java or Perl code).

Calculations in R and Excel can be combined in several different ways:

- Data from Excel can be exchanged with R via .csv files (simplest and best method),
- Excel can execute R commands using VBA scripts, and then import the R output from .csv files,
- An Excel add-in can execute R commands as Excel functions (relies on *COM* protocol, so works only for *Windows*): add-ins *BERT*, *RExcel*,
- R can modify Excel files and run Excel functions (requires packages using Java or Perl code): packages *xlsx*, *XLConnect*, *excel.link*,

```
> ### Perform calculations in R,
> ### And export to CSV files
> setwd("/Users/jerzy/Develop/lecture_slides/data")
> # Read data frame, with row names from first column
> readf <- read.csv(file="florist.csv", row.names=1)
> # Subset data frame
> readf <- readf[readf[, "type"]=="daisy", ]
> all.equal(readf, dframe)
> # Write data frame to CSV file, with row names
> write.csv(readf, file="daisies.csv")
```

Running R Code from Excel

There are several ways of performing calculations in R and exporting the outputs to Excel:

- Export data from Excel via .csv files to R, perform the calculations in R, and import the outputs back to Excel via .csv files (simplest and best method),
- Run R from Excel using VBA scripts, and exchange data via .csv files,
- Run R from Excel using an Excel add-in, and execute R commands as Excel functions (relies on the COM protocol, so works only for Windows),

```
> ### Perform calculations in R,  
> ### And export to CSV files  
> setwd("/Users/jerzy/Develop/lecture_slides/data")  
> # Read data frame, with row names from first column  
> readf <- read.csv(file="florist.csv", row.names=1)  
> # Subset data frame  
> readf <- readf[readf[, "type"]=="daisy", ]  
> all.equal(readf, dframe)  
> # Write data frame to CSV file, with row names  
> write.csv(readf, file="daisies.csv")
```

Running R Code Using VBA Scripts

An R session can be launched from Excel using a VBA script (macro).

The VBA function `shell()` executes a program by running an executable `exe` file (with extension `exe`).

A VBA script can also run an R *batch* process.

The R *batch* process can write to `.csv` files, which can then be imported into Excel.

```
' VBA macro to run R process
Sub run_r()
  Call shell("R", vbNormalFocus)
End Sub
```

```
' VBA macro to run interactive R process
Sub run_rinteractive()
  Dim script_dir As String: script_dir = "C:\Develop\R\scripts"
  Dim script_file As String: script_file = "plot_interactive.R"
  Dim filen As String: filen = "C:\Develop\R\scripts\log.txt"
  Call shell("R --vanilla < " & script_dir & script_file & ">"
End Sub
```

```
' VBA macro to run batch R process
Sub run_rbatch()
  Dim script_dir As String: script_dir = "C:\Develop\R\scripts"
  Dim script_file As String: script_file = "plot_to_file.R"
  Dim filen As String: filen = "C:\Develop\R\scripts\log.txt"
  Call shell("R --vanilla < " & script_dir & script_file & ">"
End Sub
```

BERT Excel Add-in for Running R Code

BERT is an Excel add-in which allows executing R commands as Excel functions:

<http://bert-toolkit.com/>

<http://bert-toolkit.com/bert-quick-start>

<https://github.com/sdlc/Basic-Excel-R-Toolkit/wiki>

<https://github.com/sdlc/Basic-Excel-R-Toolkit>

BERT launches its own R process from Excel.

BERT can create its own menu in the Excel add-ins tab:

After installing *BERT*, click on upper-left *Office Button*, click Excel options, on the bottom of the window choose (Manage: COM Add-ins) Go, add the COM add-in BERTRibbon2x86.dll.

BERT relies on the COM protocol, so it works only for Windows.

```
' calculate sum of Excel cells using R
R.Add(B1:D1)

' remove NAs over Excel cell range using R function
R.na_omit(F2:H4)

' calculate eigenValues of Excel matrix using R function
R.EigenValues(A1:H8)
```

Package *googlesheets* for Interacting with Google Sheets

The package *googlesheets* allows interacting with Google Sheets using R commands.

If you already have a Google account, then your personal Google Sheets can be found at:

<https://docs.google.com/spreadsheets/>

The function `gs_ls()` lists the files in Google Sheets.

The function `gs_title()` registers a Google sheet, and returns a googlesheet object.

A googlesheet object contains information (metadata) about a Google sheet, such as its name and key, but not the sheet data itself.

The function `gs_browse()` opens a Google sheet in an internet browser.

You can find online a document about [using googlesheets](#).

You can find online a document about [managing authentication tokens](#).

```
> # Install latest version of googlesheets
> devtools::install_github("jennybc/googlesheets")
> # Load package googlesheets
> library(googlesheets)
> library(dplyr)
> # Authenticate authorize R to view and manage your files
> gs_auth(new_user=TRUE)
> # List the files in Google Sheets
> googlesheets::gs_ls()
> # Register a sheet
> googsheet <- gs_title("my_data")
> # view sheet summary
> googsheet
> # List tab names in sheet
> tabv <- gs_ws_ls(googsheet)
> # Set curl options
> library(httr)
> httr::set_config(config(ssl_verifypeer=0L))
> # Read data from sheet
> gs_read(googsheet)
> # Read data from single tab of sheet
> gs_read(googsheet, ws=tabv[1])
> gs_read_csv(googsheet, ws=tabv[1])
> # Or using dplyr pipes
> googsheet %>% gs_read(ws=tabv[1])
> # Download data from sheet into file
> gs_download(googsheet, ws=tabv[1],
+             to="/Users/jerzy/Develop/lecture_slides/data/googsheet.csv")
> # Open sheet in internet browser
> gs_browse(googsheet)
```

Exception Conditions: Errors and Warnings

Exception conditions are R objects containing information about *errors* or *warnings* produced while evaluating expressions.

The function `warning()` produces a *warning* condition, but doesn't halt function execution, and returns its message to the warning handler.

The function `stop()` produces an *error* condition, halts function execution, and returns its message to the error handler.

The handling of *warning* conditions depends on the value of `options("warn")`:

- *negative* then warnings are ignored,
- *zero* then warnings are stored and printed after the top-level function has completed,
- *one* - warnings are printed as they occur,
- *two or larger* - warnings are turned into errors,

The function `suppressWarnings()` evaluates its expressions and ignores all warnings.

```
> # ?options ## Get info on global options
> getOption("warn") ## Global option for "warn"
> options("warn") ## Global option for "warn"
> getOption("error") ## Global option for "error"
> calc_sqrt <- function(inputv) {
+   # Returns its argument
+   if (inputv < 0) {
+     warning("calc_sqrt: input is negative")
+     NULL ## Return NULL for negative argument
+   } else {
+     sqrt(inputv)
+   } ## end if
+ } ## end calc_sqrt
> calc_sqrt(5)
> calc_sqrt(-1)
> options(warn=-1)
> calc_sqrt(-1)
> options(warn=0)
> calc_sqrt()
> options(warn=1)
> calc_sqrt()
> options(warn=3)
> calc_sqrt()
```

Validating Function Arguments

Argument validation consists of first determining if any arguments are *missing*, and then determining if the arguments are of the correct *type*.

An argument is *missing* when the formal argument is not bound to an actual value in the function call.

The function `missing()` returns `TRUE` if an argument is missing, and `FALSE` otherwise.

Missing arguments can be detected by:

- assigning a `NULL` default value to formal arguments and then calling `is.null()` on them,
- calling the function `missing()` on the arguments.

The argument *type* can be validated using functions such as `is.numeric()`, `is.character()`, etc.

The function `return()` returns its argument and terminates further function execution.

```
> # Function valido validates its arguments
> valido <- function(inputv=NULL) {
+ # Check if argument is valid and return double
+   if (is.null(inputv)) {
+     return("valido: input is missing")
+   } else if (is.numeric(inputv)) {
+     2*inputv
+   } else cat("valido: input not numeric")
+ } ## end valido
> valido(3)
> valido("a")
> valido()
> # valido validates arguments using missing()
> valido <- function(inputv) {
+ # Check if argument is valid and return double
+   if (missing(inputv)) {
+     return("valido: input is missing")
+   } else if (is.numeric(inputv)) {
+     2*inputv
+   } else cat("valido: input is not numeric")
+ } ## end valido
> valido(3)
> valido("a")
> valido()
```


Validating Assertions Inside Functions

If assertions about variables inside a function are FALSE, then `stop()` can be called to halt its execution.

Calling `stop()` is preferable to calling `return()`, or inserting `cat()` statements into the code.

Using `stop()` inside a function allows calling the function `traceback()`, if an error was produced.

The function `traceback()` prints the call stack, showing the function that produced the *error* condition.

`cat()` statements inside the function body provide information about the state of its variables.

```
> # valido() validates its arguments and assertions
> valido <- function(inputv) {
+ # Check if argument is valid and return double
+   if (missing(inputv)) {
+     stop("valido: input is missing")
+   } else if (!is.numeric(inputv)) {
+     cat("input =", inputv, "\n")
+     stop("valido: input is not numeric")
+   } else 2*inputv
+ } ## end valido
> valido(3)
> valido("a")
> valido()
> # Print the call stack
> traceback()
```

Validating Assertions Using stopifnot()

R provides robust validation and debugging tools through *type* validation functions, and functions `missing()`, `stop()`, and `stopifnot()`.

If the argument to function `stopifnot()` is `FALSE`, then it produces an *error* condition, and halts function execution.

`stopifnot()` is a convenience wrapper for `stop()`, and eliminates the need to use `if ()` statements.

`stopifnot()` is often used to check the validity of function arguments.

`stopifnot()` can be inserted anywhere in the function body in order to check assertions about its variables.

```
> valido <- function(inputv) {  
+ # Check argument using long form '&&' operator  
+   stopifnot(!missing(inputv) && is.numeric(inputv))  
+   2*inputv  
+ } ## end valido  
> valido(3)  
> valido()  
> valido("a")  
  
> valido <- function(inputv) {  
+ # Check argument using logical '&' operator  
+   stopifnot(!missing(inputv) & is.numeric(inputv))  
+   2*inputv  
+ } ## end valido  
> valido()  
> valido("a")
```

Validating Function Arguments and Assertions

The functions `stop()` and `stopifnot()` halt function execution and produce *error* conditions if certain assertions are FALSE.

The *type* validation functions, such as `is.numeric()`, `is.na()`, etc., and `missing()`, allow for validation of arguments and variables inside functions.

`cat()` statements can provide information about the state of variables inside a function.

`cat()` statements don't return values, so they provide information even when a function produces an error.

```
> # sumtwo() returns the sum of its two arguments
> sumtwo <- function(input1, input2) { ## Even more robust
+ # Check if at least one argument is not missing
+ stopifnot(!missing(input1) && !missing(input2))
+ # Check if arguments are valid and return sum
+ if (is.numeric(input1) && is.numeric(input2)) {
+   input1 + input2 ## Both valid
+ } else if (is.numeric(input1)) {
+   cat("input2 is not numeric\n")
+   input1 ## input1 is valid
+ } else if (is.numeric(input2)) {
+   cat("input1 is not numeric\n")
+   input2 ## input2 is valid
+ } else {
+   stop("none of the arguments are numeric")
+ }
+ } ## end sumtwo
> sumtwo(1, 2)
> sumtwo(5, 'a')
> sumtwo('a', 5)
> sumtwo('a', 'b')
> sumtwo()
```

The R Debugger Facility

The function `debug()` flags a function for future debugging, but doesn't invoke the debugger.

After a function is flagged for debugging with the call `"debug(myfun)"`, then the function call `"myfun()"` automatically invokes the debugger (browser).

When the debugger is first invoked, it prints the function code to the console, and produces a *browser* prompt: `"Browse[2]>"`.

Once inside the debugger, the user can execute the function code one command at a time by pressing the *Enter* key.

The user can examine the function arguments and variables with standard R commands, and can also change the values of objects or create new ones.

The command `"c"` executes the remainder of the function code without pausing.

The command `"Q"` exits the debugger (browser).

The call `"undebug(myfun)"` at the R prompt unflags the function for debugging.

```
> # Flag "valido" for debugging
> debug(valido)
> # Calling "valido" starts debugger
> valido(3)
> # unflag "valido" for debugging
> undebug(valido)
```

Debugging Using browser()

As an alternative to flagging a function for debugging, the user can insert the function `browser()` into the function body.

`browser()` pauses the execution of a function and invokes the debugger (browser) at the point where `browser()` was called.

Once inside the debugger, the user can execute all the same browser commands as when using `debug()`.

`browser()` is usually inserted just before the command that is suspected of producing an *error* condition.

Another alternative to flagging a function for debugging, or inserting `browser()` calls, is setting the "error" option equal to "recover".

Setting the "error" option equal to "recover" automatically invokes the debugger when an *error* condition is produced.

```
> valido <- function(inputv) {  
+   browser() ## Pause and invoke debugger  
+   # Check argument using long form '&&' operator  
+   stopifnot(!missing(inputv) && is.numeric(inputv))  
+   2*inputv  
+ } ## end valido  
> valido() ## Invokes debugger  
> options("error") ## Show default NULL "error" option  
> options(error=recover) ## Set "error" option to "recover"  
> options(error=NULL) ## Set back to default "error" option
```

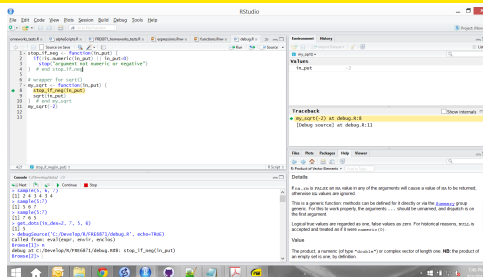
Using the Debugger in RStudio

RStudio has several built-in debugging facilities that complement those already installed in R:

- toggling breakpoints, instead of inserting `browser()` commands,
- stepping into functions,
- environment pane with environment stack, instead of calling `ls()`,
- traceback pane, instead of calling `traceback()`,

RStudio provides an online debugging tutorial:

<https://support.rstudio.com/hc/en-us/articles/205612627-Debugging-with-RStudio>



Handling Exception Conditions

The function `tryCatch()` executes functions and expressions, and handles any *exception conditions* produced when they are evaluated.

`tryCatch()` first evaluates its "expression" argument.

If no error or warning condition is produced then `tryCatch()` just returns the value of the expression.

If an exception condition is produced then `tryCatch()` invokes error and warning *handlers* and executes other expressions to provide information about the exception condition.

If a *handler* is provided to `tryCatch()` then the error is captured by the *handler*, instead of being broadcast to the console.

At the end, `tryCatch()` evaluates the expression provided to the `finally` argument.

```
> str(tryCatch) ## Get arguments of tryCatch()
> tryCatch( ## Without error handler
+ { ## Evaluate expressions
+   numv <- 101 ## Assign
+   stop("my error") ## Produce error
+ },
+ finally=print(paste0("numv = ", numv))
+ ) ## end tryCatch
>
> tryCatch( ## With error handler
+ { ## Evaluate expressions
+   numv <- 101 ## Assign
+   stop("my error") ## Produce error
+ },
+ ## Error handler captures error condition
+ error=function(msg) {
+   print(paste0("Error handler: ", msg))
+ }, ## end error handler
+ ## Warning handler captures warning condition
+ warning=function(msg) {
+   print(paste0("Warning handler: ", msg))
+ }, ## end warning handler
+ finally=print(paste0("numv = ", numv))
+ ) ## end tryCatch
```

Error Conditions in Loops

If an *error* occurs in an `apply()` loop, then the loop exits without returning any result.

`apply()` collects the values returned by the function supplied to its `FUN` argument, and returns them only after the loop is finished.

If one of the function calls produces an error, then the loop is interrupted and `apply()` exits without returning any result.

The function `tryCatch()` captures errors, allowing loops to continue after the error condition.

```
> # Apply loop without tryCatch
> apply(matrix(1:5), 1, function(numv) { ## Anonymous function
+   stopifnot(!(numv == 3)) ## Check for error
+   ## Broadcast message to console
+   cat("(cat) numv = ", numv, "\n")
+   ## Return a value
+   paste0("(return) numv = ", numv)
+ } ## end anonymous function
+ ) ## end apply
```


Exception Handling in Loops

If the body of the function supplied to the FUN argument is wrapped in tryCatch(), then the loop can finish without interruption and return its results.

The messages produced by *errors* and *warnings* can be caught by *handlers* (functions) that are supplied to tryCatch().

The *error* and *warning* messages are bound (passed) to the formal arguments of the *handler* functions that are supplied to tryCatch().

tryCatch() always evaluates the expression provided to the finally argument, even after an *error* occurs.

```
> # Apply loop with tryCatch
> apply(as.matrix(1:5), 1, function(numv) { ## Anonymous function
+   tryCatch( ## With error handler
+ { ## Body
+   stopifnot(numv != 3) ## Check for error
+   ## Broadcast message to console
+   cat("(cat) numv = ", numv, "\t")
+   ## Return a value
+   paste0("(return) numv = ", numv)
+ },
+ ## Error handler captures error condition
+ error=function(msg)
+   paste0("handler: ", msg),
+ finally=print(paste0("(finally) numv = ", numv))
+   ) ## end tryCatch
+ } ## end anonymous function
+ ) ## end apply
```

Package *data.table* for High Performance Data Management

The package *data.table* is designed for high performance data management.

The package *data.table* implements *data table* objects, which are a special type of *data frame*, and an extension of the *data frame* class.

Data tables are faster and more convenient to work with than *data frames*.

data.table functions are optimized for high performance (speed), because they are written in C++ and they perform operations by reference (in place), without copying data in memory.

Some of the attractive features of package *data.table* are:

- Syntax is analogous to SQL,
- Very fast writing and reading from files,
- Very fast sorting and merging operations,
- Subsetting using multiple logical clauses,
- Columns of type `character` are never converted to factors,

```
> # Install package data.table
> install.packages("data.table")
> # Load package data.table
> library(data.table)
> # Get documentation for package data.table
> # Get short description
> packageDescription("data.table")
> # Load help page
> help(package="data.table")
> # List all datasets in "data.table"
> data(package="data.table")
> # List all objects in "data.table"
> ls("package:data.table")
> # Remove data.table from search path
> detach("package:data.table")
```

The package *data.table* has extensive documentation:

<https://cran.r-project.org/web/packages/data.table/vignettes/datatable-intro.html>
<https://github.com/Rdatatable/data.table/wiki>

Data Table Objects

Data table objects are a special type of *data frame*, and are derived from the class `data.frame`.

Data table objects resemble databases, with columns of different types of data, and rows of records containing individual observations.

The function `data.table::data.table()` creates a *data table* object.

Data table columns can be referenced directly by their names (without quotes), and their rows can be referenced without a following comma.

When a *data table* is printed (by typing its name) then only the top 5 and bottom 5 rows are displayed (unless `getOption("datatable.print.nrows")` is less than 100).

The operator `.N` returns the number of observations (rows) in the *data table*.

Data table computations are usually much faster than equivalent R computations, but not always.

```
> # Create a data table
> library(data.table)
> dtable <- data.table::data.table(
+   col1=sample(7), col2=sample(7), col3=sample(7))
> # Print dtable
> class(dtable); dtable
> # Column referenced without quotes
> dtable[, col2]
> # Row referenced without a following comma
> dtable[2]
> # Print option "datatable.print.nrows"
> getOption("datatable.print.nrows")
> options(datatable.print.nrows=10)
> getOption("datatable.print.nrows")
> # Number of rows in dtable
> NROW(dtable)
> # Or
> dtable[, NROW(col1)]
> # Or
> dtable[, .N]
> # microbenchmark speed of data.table syntax
> library(microbenchmark)
> summary(microbenchmark(
+   dt=dtable[, .N],
+   rcode=NROW(dtable),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Writing and Reading Data Using Package *data.table*

The easiest way to share data between R and Excel is through .csv files.

The function `data.table::fread()` reads from .csv files and returns a *data table* object of class `data.table`.

Data table objects are a special type of *data frame*, and are derived from the class `data.frame`.

The function `data.table::fread()` is over 6 times faster than `read.csv()`!

The function `data.table::fwrite()` writes to .csv files over 12 times faster than the function `write.csv()`, and 300 times faster than function `cat()`!

```
> # Read a data table from CSV file
> dirn <- "/Users/jerzy/Develop/lecture_slides/data/"
> filen <- file.path(dirn, "weather_delays14.csv")
> dtable <- data.table::fread(filen)
> class(dtable); dim(dtable)
> dtable
> # fread() reads the same data as read.csv()
> all.equal(read.csv(filen),
+   setDF(data.table::fread(filen)))
> # fread() is much faster than read.csv()
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=read.csv(filen),
+   fread=setDF(data.table::fread(filen)),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
> # Write data table to file in different ways
> data.table::fwrite(dtable, file="dtable.csv")
> write.csv(dtable, file="dtable2.csv")
> cat(unlist(dtable), file="dtable3.csv")
> # microbenchmark speed of data.table::fwrite()
> summary(microbenchmark(
+   fwrite=data.table::fwrite(dtable, file="dtable.csv"),
+   write_csv=write.csv(dtable, file="dtable2.csv"),
+   cat=cat(unlist(dtable), file="dtable3.csv"),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Subsetting *Data Table* Objects

The square braces (brackets) "`[]`" operator subsets (references) the rows and columns of *data tables*.

Data table rows can be subset without a following comma.

Data table columns can be referenced directly by their names (without quotes, as if they were variables), after a comma.

Multiple *data table* columns can be referenced by passing a list of names.

The brackets "`[]`" operator is a *data.table* function, and all the commands inside the brackets "`[]`" are executed using code from the package *data.table*.

The dot `.`(`.`) operator is equivalent to the list function `list()`.

```
> # Select first five rows of dtable
> dtable[1:5]
> # Select rows with JFK flights
> jfkf <- dtable[origin=="JFK"]
> # Select rows JFK flights in June
> jfkf <- dtable[origin=="JFK" & month==6]
> # Select rows without JFK flights
> jfkf <- dtable[!(origin=="JFK")]
> # Select flights with carrier_delay
> dtable[carrier_delay > 0]
> # Select column of dtable and return a vector
> head(dtable[, origin])
> # Select column of dtable and return a dtable, not vector
> head(dtable[, list(origin)])
> head(dtable[, .(origin)])
> # Select two columns of dtable
> dtable[, list(origin, month)]
> dtable[, .(origin, month)]
> columnv <- c("origin", "month")
> dtable[, ..columnv]
> dtable[, month, origin]
> # Select two columns and rename them
> dtable[, .(orig=origin, mon=month)]
> # Select all columns except origin
> head(dtable[, !"origin"])
> head(dtable[, -"origin"])
```

Performing Computations on *Data Table* Columns

If the second argument in the brackets "`[]`" operator is a function of the columns, then the brackets return the result of the function's computations on those columns.

The second argument in the brackets "`[]`" can also be a list of functions, in which case the brackets return a vector of computations.

The brackets "`[]`" can evaluate most standard R functions, but they are executed using *data.table* code, which is usually much faster than the equivalent R functions.

The operator `.N` returns the number of observations (rows) in the *data table*.

```
> # Select flights with positive carrier_delay
> dtable[carrier_delay > 0]
> # Number of flights with carrier_delay
> dtable[, sum(carrier_delay > 0)]
> # Or standard R commands
> sum(dtable[, carrier_delay > 0])
> # microbenchmark speed of data.table syntax
> summary(microbenchmark(
+   dt=dtable[, sum(carrier_delay > 0)],
+   rcode=sum(dtable[, carrier_delay > 0]),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
> # Average carrier_delay
> dtable[, mean(carrier_delay)]
> # Average carrier_delay and aircraft_delay
> dtable[, .(carrier=mean(carrier_delay),
+   aircraft=mean(aircraft_delay))]
> # Average aircraft_delay from JFK
> dtable[origin=="JFK", mean(aircraft_delay)]
> # Number of flights from JFK
> dtable[origin=="JFK", NROW(aircraft_delay)]
> # Or
> dtable[origin=="JFK", .N]
> # In R
> sum(dtable[, origin]=="JFK")
```

Grouping *Data Table* Computations by Factor Columns

The *data table* brackets "`[]`" operator can accept three arguments: `[i, j, by]`

- `i`: the row index to select,
- `j`: a list of columns or functions on columns,
- `by`: the columns of factors to aggregate over.

The *data table* columns can be *aggregated* over categories (factors) defined by one or more columns passed to the "`by`" argument.

The "`keyby`" argument is similar to "`by`", but it sorts the output according to the categories used to group by.

Multiple *data table* columns can be referenced by passing a list of names.

The dot `.`(`)` operator is equivalent to the list function `list()`.

```
> # Number of flights from each airport
> dtable[, .N, by=origin]
> # Same, but add names to output
> dtable[, .(flights=.N), by=.(airport=origin)]
> # Number of AA flights from each airport
> dtable[carrier=="AA", .(flights=.N), by=.(airport=origin)]
> # Number of flights from each airport and airline
> dtable[, .(flights=.N), by=.(airport=origin, airline=carrier)]
> # Average aircraft_delay
> dtable[, mean(aircraft_delay)]
> # Average aircraft_delay from JFK
> dtable[origin=="JFK", mean(aircraft_delay)]
> # Average aircraft_delay from each airport
> dtable[, .(delay=mean(aircraft_delay)), by=.(airport=origin)]
> # Average and max delays from each airport and month
> dtable[, .(mean_delay=mean(aircraft_delay), max_delay=max(aircraft_delay)), by=.(airport=origin, month=month)]
+
> # Average and max delays from each airport and month
> dtable[, .(mean_delay=mean(aircraft_delay), max_delay=max(aircraft_delay)), keyby=.(airport=origin, month=month)]
+

```

Sorting *Data Table* Rows by Columns

Standard R functions can be used inside the brackets "`[]`" operator.

The function `order()` calculates the permutation index, to sort a given vector into ascending order.

The function `setorder()` sorts the rows of a *data table* by reference (in place), without copying data in memory.

`setorder()` is over 10 times faster than `order()`, because it doesn't copy data in memory.

Several brackets "`[]`" operators can be chained together to perform several consecutive computations.

```
> # Sort ascending by origin, then descending by dest
> dtables <- dtable[order(origin, -dest)]
> dtables
> # Doesn't work outside dtable
> order(origin, -dest)
> # Sort dtable by reference
> setorder(dtable, origin, -dest)
> all.equal(dtable, dtables)
> # setorder() is much faster than order()
> summary(microbenchmark(
+   order=dtable[order(origin, -dest)],
+   setorder=setorder(dtable, origin, -dest),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
> # Average aircraft_delay by month
> dtables[, .(mean_delay=mean(aircraft_delay)),
+   by=. (month=month)]
> # Chained brackets to sort output by month
> dtables[, .(mean_delay=mean(aircraft_delay)),
+   by=. (month=month)][order(month)]
```


Subsetting, Computing, and Grouping *Data Table* Objects

The special symbol `.SD` selects a subset of a *data table*.

The symbol `.SDcols` specifies the columns to select by the symbol `.SD`.

Inside the brackets `[]` operator, the `.SD` symbol can be treated as a virtual *data table*, and standard R functions can be applied to it.

The "by" argument can be used to group the outputs produced by the functions applied to the `.SD` symbol.

If the symbol `.SDcols` is not defined, then the symbol `.SD` returns the remaining columns not passed to the "by" operator.

```
> # Select weather_delay and aircraft_delay in two different ways
> dtable[1:7, .SD,
+   .SDcols=c("weather_delay", "aircraft_delay")]
> dtable[1:7, .(weather_delay, aircraft_delay)]
> # Calculate mean of weather_delay and aircraft_delay
> dtable[, sapply(.SD, mean),
+   .SDcols=c("weather_delay", "aircraft_delay")]
> sapply(dtable[, .SD,
+   .SDcols=c("weather_delay", "aircraft_delay")], mean)
> # Return origin and dest, then all other columns
> dtable[1:7, .SD, by=.(origin, dest)]
> # Return origin and dest, then weather_delay and aircraft_delay
> dtable[1:7, .SD, by=.(origin, dest),
+   .SDcols=c("weather_delay", "aircraft_delay")]
> # Return first two rows from each month
> dtable[, head(.SD, 2), by=.(month)]
> dtable[, head(.SD, 2), by=.(month),
+   .SDcols=c("weather_delay", "aircraft_delay")]
> # Calculate mean of weather_delay and aircraft_delay, grouped by
> dtable[, lapply(.SD, mean),
+   by=.(origin),
+   .SDcols=c("weather_delay", "aircraft_delay")]
> # Or simply
> dtable[, .(weather_delay=mean(weather_delay),
+   aircraft_delay=mean(aircraft_delay)),
+   by=.(origin)]
```

Modifying *Data Table* Objects by Reference

The special assignment operator `:=` allows modifying *data table* columns by reference (in place), without copying data in memory.

The computations on columns by reference can be *grouped* over categories defined by one or more columns passed to the `"by"` argument.

The computations are recycled to fit the size of each group.

The selected parts of columns can also be modified by reference, by combining the `i` and `j` arguments.

The special symbols `.SD` and `.SDcols` can be used to perform computations on several columns.

Modifying by reference is several times faster than standard R assignment.

```
> # Add tot_delay column
> dttable[, tot_delay := (carrier_delay + aircraft_delay)]
> head(dttable, 4)
> # Delete tot_delay column
> dttable[, tot_delay := NULL]
> # Add max_delay column grouped by origin and dest
> dttable[, max_delay := max(aircraft_delay), by=.(origin, dest)]
> dttable[, max_delay := NULL]
> # Add date and tot_delay columns
> dttable[, c("date", "tot_delay") :=
+   list(paste(month, day, year, sep="/"),
+         (carrier_delay + aircraft_delay))]
> # Modify select rows of tot_delay column
> dttable[month == 12, tot_delay := carrier_delay]
> dttable[, c("date", "tot_delay") := NULL]
> # Add several columns
> dttable[, c("max_carrier", "max_aircraft") := lapply(.SD, max),
+   by=.(origin, dest),
+   .SDcols=c("carrier_delay", "aircraft_delay")]
> # Remove columns
> dttable[, c("max_carrier", "max_aircraft") := NULL]
> # Modifying by reference is much faster than standard R
> summary(microbenchmark(
+   dt=dttable[, tot_delay := (carrier_delay + aircraft_delay)],
+   rcode=(dttable[, "tot_delay"] <- dttable[, "carrier_delay"] + dt
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Adding keys to *Data Tables* for Fast Binary Search

The key of a *data table* is analogous to the row indices of a *data frame*, and it determines the ordering of its rows.

The function `data.table::setkey()` adds a *key* to a *data table*, and sorts the *data table* rows by reference according to the key.

`setkey()` creates the *key* from one or more columns of the *data frame*.

Subsetting rows using a *key* can be several times faster than standard R.

```
> # Add a key based on the "origin" column
> setkey(dtable, origin)
> haskey(dtable)
> key(dtable)
> # Select rows with LGA using the key
> dtable["LGA"]
> all.equal(dtable["LGA"], dtable[origin == "LGA"])
> # Select rows with LGA and JFK using the key
> dtable[c("LGA", "JFK")]
> # Add a key based on the "origin" and "dest" columns
> setkey(dtable, origin, dest)
> key(dtable)
> # Select rows with origin from JFK and MIA
> dtable[c("JFK", "MIA")]
> # Select rows with origin from JFK and dest to MIA
> dtable[.("JFK", "MIA")]
> all.equal(dtable[.("JFK", "MIA")],
+   dtable[origin == "JFK" & dest == "MIA"])
> # Selecting rows using a key is much faster than standard R
> summary(microbenchmark(
+   with_key=dtable[.("JFK", "MIA")],
+   standard_r=dtable[origin == "JFK" & dest == "MIA"],
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Coercing *Data Table* Objects Into *Data Frames*

The functions `data.table::setDT()` and `data.table::setDF()` coerce *data frames* to *data tables*, and vice versa.

The *set* functions `data.table::set*()` perform their operations by reference (in place), without returning any values or copying data to a new memory location, which makes them very fast.

Data table objects can also be coerced into *data frames* using the function `as.data.frame()`, but it's much slower because it makes copies of data.

```
> # Create data frame and coerce it to data table
> dtable <- data.frame(col1=sample(7), col2=sample(7), col3=sample(7))
> class(dtable); dtable
> data.table::setDT(dtable)
> class(dtable); dtable
> # Coerce dtable into data frame
> data.table::setDF(dtable)
> class(dtable); dtable
> # Or
> dtable <- data.table::as.data.frame.data.table(dtable)
> # SetDF() is much faster than as.data.frame()
> summary(microbenchmark(
+   asdataframe=data.table::as.data.frame.data.table(dtable),
+   setDF=data.table::setDF(dtable),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Coercing xts Time Series Into *Data Tables*

An xts time series can be coerced into a *data table* by first coercing it into a *data frame* and then into a *data table* using the function `data.table::setDT()`.

But then the time index of the xts series is coerced into strings, not dates.

An xts time series can also be coerced directly into a *data table* using the function `data.table::as.data.table()`.

```
> # Coerce xts to a data frame
> pricev <- rutils::etfenv$VTI
> class(pricev); head(pricev)
> pricev <- as.data.frame(pricev)
> class(pricev); head(pricev)
> # Coerce data frame to a data table
> data.table::setDT(pricev, keep.rownames=TRUE)
> class(pricev); head(pricev)
> # Dates are coerced to strings
> sapply(pricev, class)
> # Coerce xts directly to a data table
> dtable <- as.data.table(rutils::etfenv$VTI,
+   keep.rownames=TRUE)
> class(dtable); head(dtable)
> # Dates are not coerced to strings
> sapply(dtable, class)
> all.equal(pricev, dtable, check.attributes=FALSE)
```

Package *fst* for High Performance Data Management

The package *fst* provides functions for very fast writing and reading of *data frames* from *compressed binary files*.

The package *fst* writes to *compressed binary files* in the *fst* fast-storage format.

The package *fst* uses the LZ4 and ZSTD compression algorithms, and utilizes multithreaded (parallel) processing on multiple CPU cores.

The package *fst* has extensive documentation:

<http://www.fstpackage.org/>

```
> # Install package fst
> install.packages("fst")
> # Load package fst
> library(fst)
> # Get documentation for package fst
> # Get short description
> packageDescription("fst")
> # Load help page
> help(package="fst")
> # List all datasets in "fst"
> data(package="fst")
> # List all objects in "fst"
> ls("package:fst")
> # Remove fst from search path
> detach("package:fst")
```

Writing and Reading Data Using Package *fst*

The package *fst* allows very fast writing and reading of *data frames* from *compressed binary files* in the *fst* fast-storage format.

The function `fst::write_fst()` writes to *.fst* files over 10 times faster than the function `write.csv()`, and 300 times faster than function `cat()` write to *.csv* files!

The function `fst::fread()` reads from *.fst* files over 10 times faster than the function `read.csv()` from *.csv* files!

```
> # Read a data frame from CSV file
> dirn <- "/Users/jerzy/Develop/lecture_slides/data/"
> filen <- file.path(dirn, "weather_delays14.csv")
> data.table::setDF(dframe)
> class(dframe); dim(dframe)
> # Write data frame to .fst file in different ways
> fst::write_fst(dframe, path="dframe.fst")
> write.csv(dframe, file="dframe2.csv")
> # microbenchmark speed of fst::write_fst()
> library(microbenchmark)
> summary(microbenchmark(
+   fst=fst::write_fst(dframe, path="dframe.csv"),
+   write_csv=write.csv(dframe, file="dframe2.csv"),
+   cat=cat(unlist(dframe), file="dframe3.csv"),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
> # fst::read_fst() reads the same data as read.csv()
> all.equal(read.csv(filen),
+   fst::read_fst("dframe.fst"))
> # fst::read_fst() is 10 times faster than read.csv()
> summary(microbenchmark(
+   fst=fst::read_fst("dframe.fst"),
+   read_csv=read.csv(filen),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Random Access to Large Data Files

The package *fst* allows *random access* to very large *data frames* stored in compressed data files in the *.fst* format.

Data frames can be accessed *randomly* by loading only the selected rows and columns into memory, without fully loading the whole data frame.

function `fst::fst()` reads an *.fst* file and returns an *fst_table* reference object (pointer) to the data, without loading the whole data into memory.

The *fst_table* reference provides access to the data similar to a regular *data frame*, but it requires only a small amount of memory because the data isn't loaded into memory.

```
> # Coerce TAQ xts to a data frame
> library(HighFreq)
> taq <- HighFreq::SPY_TAQ
> taq <- as.data.frame(taq)
> class(taq)
> # Coerce data frame to a data table
> data.table::setDT(taq, keep.rownames=TRUE)
> class(taq); head(taq)
> # Get memory size of data table
> format(object.size(taq), units="MB")
> # Save data table to .fst file
> fst::write_fst(taq, path="/Users/jerzy/Develop/data/taq.fst")
> # Create reference to .fst file similar to a data frame
> refst <- fst::fst("/Users/jerzy/Develop/data/taq.fst")
> class(refst)
> # Memory size of reference to .fst is very small
> format(object.size(refst), units="MB")
> # Get sizes of all objects in workspace
> sort(sapply(mget(ls()), object.size))
> # Reference to .fst can be treated similar to a data table
> dim(taq); dim(refst)
> fst::print.fst_table(refst)
> # Subset reference to .fst just like a data table
> refst[1e4:(1e4+5), ]
```


Writing Text Strings

The function `cat()` concatenates strings and writes them to standard output or to files.

`cat()` parses its argument character string and its escape sequences ("`\`"), but doesn't return a value.

The function `print()` doesn't interpret its argument, and simply prints it to standard output and invisibly returns it.

Typing the name of an object in R implicitly calls `print()` on that object.

The function `save()` writes objects to compressed binary `.RData` files.

```
> cat("Enter\ttab") ## Cat() parses backslash escape sequences
> print("Enter\ttab")
>
> textv <- print("hello")
> textv ## Print() returns its argument
>
> # Create string
> textv <- "Title: My Text\nSome numbers: 1,2,3...\nRprofile files"
>
> cat(textv, file="mytext.txt") ## Write to text file
>
> cat("Title: My Text", ## Write several lines to text file
+     "Some numbers: 1,2,3...",
+     "Rprofile files contain code executed at R startup",
+     file="mytext.txt", sep="\n")
>
> save(textv, file="mytext.RData") ## Write to binary file
```

Displaying Numeric Data

The function `print()` displays numeric data objects, with the number of digits given by the global option `"digits"`.

The function `sprintf()` returns strings formatted from text strings and numeric data.

```
> print(pi)
[1] 3.14
> print(pi, digits=10)
[1] 3.141592654
> getOption("digits")
[1] 3
> foo <- 12
> bar <- "weeks"
> sprintf("There are %i %s in the year", foo, bar)
[1] "There are 12 weeks in the year"
```

Reading Text from Files

The function `scan()` reads text or data from a file and returns it as a vector or a list.

The function `readLines()` reads lines of text from a connection (file or console), and returns them as a vector of character strings.

The function `readline()` reads a single line from the console, and returns it as a character string.

The function `file.show()` reads text or data from a file and displays in editor.

```
> # Read text from file
> scan(file="mytext.txt", what=character(), sep="\n")
>
> # Read lines from file
> readLines(con="mytext.txt")
>
> # Read text from console
> inputv <- readline("Enter a number: ")
> class(inputv)
> # Coerce to numeric
> inputv <- as.numeric(inputv)
>
> # Read text from file and display in editor:
> # file.show("mytext.txt")
> # file.show("mytext.txt", pager="")
```

Writing and Reading *Data Frames* from Text Files

The functions `write.table()` and `read.table()` write and read *data frames* from text files.

`write.table()` coerces objects to *data frames* before it writes them.

`read.table()` returns a *data frame*, without coercing non-numeric values to factors (so no need for the option `stringsAsFactors=FALSE`).

`write.table()` and `read.table()` can be used to write and read matrices from text files, but they have to be coerced back to matrices.

`write.table()` and `read.table()` are inefficient for very large data sets.

```
> setwd("/Users/jerzy/Develop/lecture_slides/data")
> dframe <- data.frame(type=c("rose", "daisy", "tulip"),
+   color=c("red", "white", "yellow"),
+   price=c(1.5, 0.5, 1.0),
+   row.names=c("flower1", "flower2", "flower3")) ## end data.frame
> matv <- matrix(sample(1:12), ncol=3,
+   dimnames=list(NULL, c("col1", "col2", "col3")))
> rownames(matv) <- paste("row", 1:NROW(matv), sep="")
> # Write data frame to text file, and then read it back
> write.table(dframe, file="florist.txt")
> readf <- read.table(file="florist.txt")
> readf ## A data frame
> all.equal(readf, dframe)
>
> # Write matrix to text file, and then read it back
> write.table(matv, file="matrix.txt")
> readmat <- read.table(file="matrix.txt")
> readmat ## write.table() coerced matrix to data frame
> class(readmat)
> all.equal(readmat, matv)
> # Coerce from data frame back to matrix
> readmat <- as.matrix(readmat)
> class(readmat)
> all.equal(readmat, matv)
```

Copying *Data Frames* Between the *clipboard* and R

Data frames stored in the *clipboard* can be copied into R using the function `read.table()`.

Data frames in R can be copied into the *clipboard* using the function `write.table()`.

This allows convenient copying of *data frames* between R and Excel.

Data frames can also be manipulated directly in the R spreadsheet-style data editor.

Copying and pasting between the *clipboard* and R works well on Windows, but not on MacOS. There are some workarounds for MacOS:

Copy-paste-between-R-and-clipboard

```
> # Create a data frame
> dframe <- data.frame(small=c(3, 5), medium=c(9, 11), large=c(15, 17))
>
> # Launch spreadsheet-style data editor
> dframe <- edit(dframe)
>
> # Copy the data frame to clipboard
> write.table(x=dframe, file="clipboard", sep="\t")
>
> # Wrapper function for copying data frame from R into clipboard
> # by default, data is tab delimited, with a header
> write_clip <- function(data, namev=FALSE, col.names=TRUE, ...) {
+   write.table(x=data, file="clipboard", sep="\t",
+     row.names=namev, col.names=col.names, ...)
+ } ## end write_clip
>
> write_clip(data=dframe)
>
> # Wrapper function for copying data frame from clipboard into R
> # by default, data is tab delimited, with a header
> read_clip <- function(file="clipboard", sep="\t", header=TRUE, ...) {
+   read.table(file=file, sep=sep, header=header, ...)
+ } ## end read_clip
>
> dframe <- read.table("clipboard", header=TRUE)
> dframe <- read_clip()
```

Writing and Reading *Data Frames* From .csv Files

The easiest way to share data between R and Excel is through .csv files.

The functions `write.csv()` and `read.csv()` write and read *data frames* from .csv format files.

The functions `write.csv()` and `read.csv()` write and read *data frames* from .csv format files.

These functions are *wrappers* for `write.table()` and `read.table()`.

`read.csv()` doesn't coerce non-numeric values to factors, so no need for the option `stringsAsFactors=FALSE`.

`read.csv()` reads row names as an extra column, unless the `row.names=1` argument is used.

The argument "row.names" accepts either the number or the name of the column containing the row names.

The `*.csv()` functions are very inefficient for large data sets.

```
> # Write data frame to CSV file, and then read it back
> write.csv(dframe, file="florist.csv")
> readf <- read.csv(file="florist.csv")
> readf ## the row names are read in as extra column
> # Restore row names
> rownames(readf) <- readf[, 1]
> readf <- readf[, -1] ## Remove extra column
> readf
> all.equal(readf, dframe)
> # Read data frame, with row names from first column
> readf <- read.csv(file="florist.csv", row.names=1)
> readf
> all.equal(readf, dframe)
```

Writing and Reading *Data Frames* From .csv Files (cont.)

The functions `write.csv()` and `read.csv()` can write and read *data frames* from .csv format files *without using row names*.

Row names can be omitted from the output file by calling `write.csv()` with the argument `row.names=FALSE`.

```
> # Write data frame to CSV file, without row names
> write.csv(dframe, row.names=FALSE, file="florist.csv")
> readf <- read.csv(file="florist.csv")
> readf ## A data frame without row names
> all.equal(readf, dframe)
```

Reading Data From Very Large .csv Files

Data from very large .csv files can be read in small chunks instead of all at once.

The function `file()` opens a connection to a file or an internet website URL.

The function `read.csv()` with the argument "nrows" reads only the specified number of rows from a connection and returns a *data frame*. The connection pointer is reset to the next row.

The function `read.csv()` with the argument "nrows" allows reading data sequentially from very large files that wouldn't fit into memory.

```
> # Open a read connection to a file
> filecon = file("/Users/jerzy/Develop/lecture_slides/data/etf_prices.csv")
> # Read the first 10 rows
> data10 <- read.csv(filecon, nrows=10)
> # Read another 10 rows
> data20 <- read.csv(filecon, nrows=10, header=FALSE)
> colnames(data20) <- colnames(data10)
> # Close the connection to the file
> close(filecon)
> # Open a read connection to a file
> filecon = file("/Users/jerzy/Develop/lecture_slides/data/etf_prices.csv")
> # Read the first 1000 rows
> data10 <- read.csv(filecon, nrows=1e3)
> colv <- colnames(data10)
> # Write to a file
> countv <- 1
> write.csv(data10, paste0("/Users/jerzy/Develop/data/temp/etf_prices.csv"),
+           as.csv = FALSE, append = TRUE)
> # Read remaining rows in a loop 10 rows at a time
> # Can produce error without getting to end of file
> while (isOpen(filecon)) {
+   datav <- read.csv(filecon, nrows=1e3)
+   colnames(datav) <- colv
+   write.csv(datav, paste0("/Users/jerzy/Develop/data/temp/etf_prices.csv"),
+             as.csv = FALSE, append = TRUE)
+   countv <- countv + 1
+ } ## end while
```


Writing and Reading Matrices From .csv Files

The functions `write.csv()` and `read.csv()` can write and read matrices from .csv format files.

If row names can be omitted in the output file, then `write.csv()` can be called with argument `row.names=FALSE`.

If the input file doesn't contain row names, then `read.csv()` can be called without the "row.names" argument.

```
> # Write matrix to csv file, and then read it back
> write.csv(mtv, file="matrix.csv")
> readmat <- read.csv(file="matrix.csv", row.names=1)
> readmat ## Read.csv() reads matrix as data frame
> class(readmat)
> readmat <- as.matrix(readmat) ## Coerce to matrix
> all.equal(readmat, mtv)
> write.csv(mtv, row.names=FALSE,
+   file="matrix_ex_rows.csv")
> readmat <- read.csv(file="matrix_ex_rows.csv")
> readmat <- as.matrix(readmat)
> readmat ## A matrix without row names
> all.equal(readmat, mtv)
```

Writing and Reading Matrices (cont.)

There are several ways of writing and reading matrices from .csv files, with tradeoffs between simplicity, data size, and speed.

The function `write.matrix()` writes a matrix to a text file, without its row names.

`write.matrix()` is part of package *MASS*.

The advantage of function `scan()` is its speed, but it doesn't handle row names easily.

Removing row names simplifies the writing and reading of matrices.

The function `readLines` reads whole lines and returns them as single strings.

```
> setwd("/Users/jerzy/Develop/lecture_slides/data")
> library(MASS) ## Load package "MASS"
> # Write to CSV file by row - it's very SLOW!!!
> MASS::write.matrix(matv, file="matrix.csv", sep=",")
> # Read using scan() and skip first line with colnames
> readmat <- scan(file="matrix.csv", sep=",", skip=1,
+   what=numeric())
> # Read colnames
> colv <- readLines(con="matrix.csv", n=1)
> colv ## this is a string!
> # Convert to char vector
> colv <- strsplit(colv, split=",")[[1]]
> readmat ## readmat is a vector, not matrix!
> # Coerce by row to matrix
> readmat <- matrix(readmat, ncol=NROW(colv), byrow=TRUE)
> # Restore colnames
> colnames(readmat) <- colv
> readmat
> # Scan() is a little faster than read.csv()
> library(microbenchmark)
> summary(microbenchmark(
+   read_csv=read.csv("matrix.csv"),
+   scan=scan(file="matrix.csv", sep=",",
+     skip=1, what=numeric()),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Reading Matrices Containing Bad Data

Very often data that is read from external sources contains elements with bad data.

An example of bad data are character strings within sets of numeric data.

Columns of numeric data that contain strings are coerced to character or factor, when they're read by `read.csv()`.

The function `as.numeric()` coerces complex data objects into numeric vectors, and removes all their *attributes*.

`as.numeric()` coerces strings that don't represent numbers into NA values.

```
> # Read data from a csv file, including row names
> matv <- read.csv(file="data/matrix_bad.csv", row.names=1)
> matv
> class(matv)
> # Columns with bad data are character or factor
> sapply(matv, class)
> # Coerce character column to numeric
> matv$col2 <- as.numeric(matv$col2)
> # Or
> # Copy row names
> namev <- rownames(matv)
> # sapply loop over columns and coerce to numeric
> matv <- sapply(matv, as.numeric)
> # Restore row names
> rownames(matv) <- namev
> # Replace NAs with zero
> matv[is.na(matv)] <- 0
> # matrix without NAs
> matv
```

Writing and Reading Time Series From *Text* Files

The package *zoo* contains functions `write.zoo()` and `read.zoo()` for writing and reading *zoo* time series from `.txt` and `.csv` files.

The functions `write.zoo()` and `read.zoo()` are *wrappers* for `write.table()` and `read.table()`.

The function `write.zoo()` writes the *zoo* series index as a character string in quotations "", to make it easier to read (parse) by `read.zoo()`.

Users may also directly use `write.table()` and `read.table()`, instead of `write.zoo()` and `read.zoo()`.

```
> library(zoo) ## Load package zoo
> # Create zoo with Date index
> datev <- seq(from=as.Date("2013-06-15"), by="day",
+             length.out=100)
> pricev <- zoo(rnorm(NROW(datev)), order.by=datev)
> head(pricev, 3)
> # Write zoo series to text file, and then read it back
> write.zoo(pricev, file="pricev.txt")
> pricezoo <- read.zoo("pricev.txt") ## Read it back
> all.equal(pricezoo, pricev)
> # Perform the same using write.table() and read.table()
> # First coerce pricev into data frame
> dframe <- as.data.frame(pricev)
> dframe <- cbind(datev, dframe)
> # Write pricev to text file using write.table
> write.table(dframe, file="pricev.txt",
+             row.names=FALSE, col.names=FALSE)
> # Read data frame from file
> pricezoo <- read.table(file="pricev.txt")
> sapply(pricezoo, class) ## A data frame
> # Coerce data frame into pricev
> pricezoo <- zoo::zoo(
+   drop(as.matrix(pricezoo[, -1])),
+   order.by=as.Date(pricezoo[, 1]))
> all.equal(pricezoo, pricev)
```

Writing and Reading Time Series From .csv Files

By default the functions `zoo::write.zoo()` and `zoo::read.zoo()` write data in *space*-delimited text format, but they can also write to *comma*-delimited .csv files by passing the parameter `sep=","`.

Single column *zoo* time series usually don't have a dimension attribute, and they don't have a column name, unlike multi-column *zoo* time series, and this can cause hard to detect bugs.

It's best to always pass the argument `"col.names=TRUE"` to the function `write.zoo()`, to make sure it writes a column name for a single column *zoo* time series.

Reading a .csv file containing a single column of data using the function `read.zoo()` produces a *zoo* time series with a `NULL` dimension, unless the argument `"drop=FALSE"` is passed to `read.zoo()`.

Users may also directly use `write.table()` and `read.table()`, instead of `write.zoo()` and `read.zoo()`.

```
> # Write zoo series to CSV file, and then read it back
> write.zoo(pricev, file="zooseries.csv", sep=",", col.names=TRUE)
> pricezoo <- read.zoo(file="zooseries.csv",
+   header=TRUE, sep=",", drop=FALSE)
> all.equal(pricev, drop(pricezoo))
```

Writing and Reading Time Series With *Date-time* Index

The function `write.zoo()` writes *zoo* time series into *.csv* files, but it doesn't format the time at midnight properly.

The function `write.table()` writes *zoo* time series into *.csv* files, and it formats the time at midnight properly.

If the index of a *zoo* time series is a *date-time*, then `write.zoo()` writes the date and time fields as character strings separated by a *space* between them, inside quotations `" "`.

The functions `read.csv.zoo()` and `read.zoo()` read *zoo* time series from *.csv* files.

Very often *.csv* files contain custom *date-time* formats, which need to be passed as parameters into `read.zoo()` for proper formatting.

The "FUN" argument of `read.zoo()` accepts a function for coercing the date and time columns of the input data into a *date-time* object suitable for the *zoo* index.

The function `as.POSIXct()` coerces character strings into *POSIXct date-time* objects.

```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Create zoo with POSIXct date-time index
> datev <- seq(from=as.POSIXct("2014-07-14"),
+             by="hour", length.out=100)
> zooseries <- zoo(rnorm(NROW(datev)), order.by=datev)
> head(zooseries, 3)
> # Write zoo series to CSV file using write.zoo()
> write.zoo(zooseries, file="zooseries.csv", sep=",", col.names=TRUE)
> # Read from CSV file using read.csv.zoo() - doesn't work
> zooread <- read.csv.zoo(file="zooseries.csv", header=FALSE,
+ format="%Y-%m-%d %H:%M:%S", tz="America/New_York")
> # Read from CSV file using read.zoo() - error
> zooread <- read.zoo(file="zooseries.csv", header=FALSE,
+ sep=",", FUN=as.POSIXct, format="%Y-%m-%d %H:%M:%S")
> # Write zoo series to CSV file using write.table()
> write.table(zooseries, file="zooseries.csv", sep=",",
+             row.names=TRUE, col.names=FALSE)
> # Read from CSV file using read.zoo() with format argument
> zooread <- read.zoo(file="zooseries.csv", header=FALSE,
+ sep=",", FUN=as.POSIXct, format="%Y-%m-%d %H:%M:%S")
> all.equal(zooseries, zooread) ## Works
> # Coerce zoo series into data frame with custom date format
> dframe <- as.data.frame(zooseries)
> rownames(dframe) <- format(index(zooseries), format="%m-%d-%Y %H:%M:%S")
> # Write zoo series to csv file using write.table
> write.table(dframe, file="zooseries.csv", sep=",",
+             row.names=TRUE, col.names=FALSE)
> # Read from CSV file using read.zoo()
> zooread <- read.zoo(file="zooseries.csv", header=FALSE, sep=",",
+ FUN=as.POSIXct, format="%m-%d-%Y %H:%M:%S")
> all.equal(zooseries, zooread) ## Works
> # Or using read.csv.zoo()
> zooread <- read.csv.zoo(file="zooseries.csv", header=FALSE,
+ format="%m-%d-%Y %H:%M:%S", tz="America/New_York")
> head(zooread, 3)
> all.equal(zooseries, zooread, check.attributes=FALSE) ## Works
```

Reading Time Series With Numeric *Date-time* Index

If the index of a time series is numeric (representing the *moment of time*, either as the number of days or seconds), then it must be coerced to a proper *date-time* class.

A convenient way of reading time series with a numeric index is by using `read.table()`, and then coercing the *data frame* into a time series.

The function `as.POSIXct.numeric()` coerces a numeric value representing the *moment of time* into a `POSIXct` *date-time*, equal to the *clock time* in the local *time zone*.

```
> # Read time series from CSV file, with numeric date-time
> datazoo <- read.table(file="/Users/jerzy/Develop/lecture_slides/d
+   header=TRUE, sep=",")
> # A data frame
> class(datazoo)
> sapply(datazoo, class)
> # Coerce data frame into xts series
> datazoo <- xts::xts(as.matrix(datazoo[, -1]),
+   order.by=as.POSIXct.numeric(datazoo[, 1], tz="America/New_York",
+   origin="1970-01-01"))
> # An xts series
> class(datazoo)
> head(datazoo, 3)
```

Passing Arguments to the save() Function

The function `save()` writes objects to a binary file.

Object names can be passed into `save()` either through the `"..."` argument, or the `"list"` argument.

Objects passed through the `"..."` argument are not evaluated, so they must be either object names or character strings.

Object names aren't surrounded by quotes `"`, while character strings that represent object names are surrounded by quotes `"`.

Objects passed through the `"list"` argument are evaluated, so they may be variables containing character strings.

```
> var1 <- 1; var2 <- 2
> ls() ## List all objects
> ls()[1] ## List first object
> args(save) ## List arguments of save function
> # Save "var1" to a binary file using string argument
> save("var1", file="my_data.RData")
> # Save "var1" to a binary file using object name
> save(var1, file="my_data.RData")
> # Save multiple objects
> save(var1, var2, file="my_data.RData")
> # Save first object in list by passing to "..." argument
> # ls()[1] is not evaluated
> save(ls()[1], file="my_data.RData")
> # Save first object in list by passing to "list" argument
> save(list=ls()[1], file="my_data.RData")
> # Save whole list by passing it to the "list" argument
> save(list=ls(), file="my_data.RData")
```


Writing and Reading Lists of Objects

The function `load()` reads data from `.RData` files, and *invisibly* returns a vector of names of objects created in the workspace.

The vector of names can be used to manipulate the objects in loops, or to pass them to functions.

```
> rm(list=ls()) ## Delete all objects in workspace
> # Load objects from file
> loadobj <- load(file="my_data.RData")
> loadobj ## vector of loaded objects
> ls() ## List objects
> # Assign new values to objects in global environment
> sapply(loadobj, function(symboln) {
+   assign(symboln, runif(1), envir=globalenv())
+ }) ## end sapply
> ls() ## List objects
> # Assign new values to objects using for loop
> for (symboln in loadobj) {
+   assign(symboln, runif(1))
+ } ## end for
> ls() ## List objects
> # Save vector of objects
> save(list=loadobj, file="my_data.RData")
> # Remove only loaded objects
> rm(list=loadobj)
> # Remove the object "loadobj"
> rm(loadobj)
```

Saving Output of R to a File

The function `sink()` diverts R text output (excluding graphics) to a file, or ends the diversion.

Remember to call `sink()` to end the diversion!

The function `pdf()` diverts graphics output to a *pdf* file (text output isn't diverted), in vector graphics format.

The functions `png()`, `jpeg()`, `bmp()`, and `tiff()` divert graphics output to graphics files (text output isn't diverted).

The function `dev.off()` ends the diversion.

```
> sink("sinkdata.txt")# Redirect text output to file
>
> cat("Redirect text output from R\n")
> print(runif(10))
> cat("\nEnd data\nbye\n")
>
> sink() ## turn redirect off
>
> pdf("Rgraph.pdf", width=7, height=4) ## Redirect graphics to pdf
>
> cat("Redirect data from R into pdf file\n")
> myvar <- seq(-2*pi, 2*pi, len=100)
> plot(x=myvar, y=sin(myvar), main="Sine wave",
+      xlab="", ylab="", type="l", lwd=2, col="red")
> cat("\nEnd data\nbye\n")
>
> dev.off() ## turn pdf output off
>
> png("r_plot.png") ## Redirect graphics output to png file
>
> cat("Redirect graphics from R into png file\n")
> plot(x=myvar, y=sin(myvar), main="Sine wave",
+      xlab="", ylab="", type="l", lwd=2, col="red")
> cat("\nEnd data\nbye\n")
>
> dev.off() ## turn png output off
```

Homework Assignment

No homework!

